MODULE 3

DATA VISUALIZATION

Detailed Study Notes • Python • Matplotlib • Pandas • Seaborn

Syllabus covered

- A brief Matplotlib API primer
- Plotting with Pandas and Seaborn
- Line plots and bar plots
- Histograms and density plots
- Scatter plots
- Facet grids and categorical data
- Other Python visualization tools

Beginner-friendly explanations, syntax, examples, interpretation tips, and revision points.

How to use these notes

These notes follow the topics in the Module 3 syllabus. Read the concepts first, run the code examples in a Python environment, and then revise the comparison tables and practice questions. The examples use small, invented datasets so that each plot is easy to understand.

Install the libraries

```
python -m pip install matplotlib pandas seaborn
```

In a notebook, charts usually appear below the code cell. In a Python script, call `plt.show()` to display a chart. If your computer uses the command `python3`, run `python3 -m pip install matplotlib pandas seaborn`.

Learning outcomes

• Choose a suitable chart for a question or dataset.

• Create and customize charts with Matplotlib.

• Plot directly from a Pandas Series or DataFrame.

• Use Seaborn for statistical and categorical visualizations.

• Explain axes, labels, legends, bins, density, hue, facets, and chart limitations.

1. Data visualization: foundation

Data visualization means representing data visually—using charts, plots, and graphs—to make patterns, comparisons, distributions, and relationships easier to understand.

Why visualize data?

• Understand a dataset quickly before modeling or reporting.

• Compare values across categories or time periods.

• Find trends, clusters, unusual observations, and possible outliers.

• Communicate results to people who may not want to inspect raw tables.

• Check data quality—for example, unexpected gaps, impossible values, or skewed distributions.

Match the question to the chart

Question	Useful plot	Example
How does something change over time?	Line plot	Daily temperature
Which category is larger?	Bar plot	Sales by product
How are values distributed?	Histogram / density plot	Exam marks
Are two numeric variables related?	Scatter plot	Study hours vs marks
How does a pattern differ across groups?	Facet grid	Sales trend by region
How do numeric values vary by category?	Box / violin / strip plot	Salary by department

2. A brief Matplotlib API primer

Matplotlib is a widely used Python library for creating static, animated, and interactive visualizations. Its `pyplot` interface provides familiar functions such as `plot()`, `bar()`, and `hist()`.

2.1 Import and basic structure

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 3, 7, 6]

plt.plot(x, y, marker="o", label="Values")
plt.title("A simple line plot")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

What each function does:

• `plt.plot(x, y)`: draws a line connecting the supplied points.

• `marker='o'`: places a circular marker at each data point.

• `title()`: adds a title; `xlabel()` and `ylabel()` label the axes.

• `grid()`: adds reference lines that can help read values.

• `legend()`: displays labels for plotted series.

• `tight_layout()`: adjusts spacing to reduce clipped labels.

• `show()`: displays the finished figure in a script.

2.2 Figure and Axes: object-oriented API

Matplotlib organizes a chart around a **Figure** (the whole canvas) and one or more **Axes** (the actual plotting areas). The object-oriented API is recommended when a figure has multiple plots or needs careful customization.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot([1, 2, 3], [3, 5, 4], marker="o")
ax.set_title("Weekly values")
ax.set_xlabel("Week")
ax.set_ylabel("Value")
ax.grid(True, alpha=0.3)
fig.tight_layout()
plt.show()
```

• `fig` represents the complete figure.

• `ax` represents one plotting area.

• `plt.subplots()` creates the Figure and Axes together.

• Use `ax.set_title()`, `ax.set_xlabel()`, and `ax.set_ylabel()` for an Axes object.

2.3 Multiple plots in one figure

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].plot([1, 2, 3], [2, 4, 3], marker="o")
axes[0].set_title("Line plot")

axes[1].bar(["A", "B", "C"], [5, 3, 7])
axes[1].set_title("Bar plot")

fig.tight_layout()
plt.show()
```

Here, `1, 2` means one row and two columns of plots. With multiple Axes, customize each one through `axes[0]`, `axes[1]`, and so on.

2.4 Common customization options

• Color: `color='teal'`.

• Line style: `linestyle='--'` for a dashed line.

• Line width: `linewidth=2`.

• Marker: `marker='s'` for squares, `marker='o'` for circles.

• Transparency: `alpha=0.5`.

• Size: `figsize=(8, 5)`.

• Save: `fig.savefig('chart.png', dpi=300, bbox_inches='tight')`.

Good practice: Give every chart a meaningful title, label both axes, include units where relevant, and use a legend only when it helps the reader.

3. Plotting with Pandas

Pandas is a Python library for working with tabular data. A DataFrame stores rows and columns; a Series stores one column or a one-dimensional sequence. Pandas plotting methods use Matplotlib behind the scenes, so Matplotlib customization can still be applied.

3.1 Create a DataFrame

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "month": ["Jan", "Feb", "Mar", "Apr"],
    "sales": [120, 150, 135, 180],
    "cost": [80, 90, 95, 110]
})
```

3.2 Basic Pandas plotting

```
df.plot(x="month", y="sales", kind="line", marker="o",
        title="Monthly sales")
plt.ylabel("Sales (units)")
plt.tight_layout()
plt.show()
```

The `kind` argument chooses the chart type. Common values include `line`, `bar`, `barh`, `hist`, `box`, `area`, and `scatter` (scatter generally requires x and y columns).

3.3 Plot multiple columns

```
df.plot(x="month", y=["sales", "cost"], marker="o",
        title="Sales and cost by month")
plt.ylabel("Amount")
plt.tight_layout()
plt.show()
```

Pandas automatically uses different visual styles for the two columns and adds a legend. Always check that the columns share a meaningful scale before comparing them on one axis.

3.4 Histogram and bar chart using Pandas

```
# Histogram: distribution of a numeric column
marks = pd.Series([45, 50, 55, 60, 61, 65, 70, 72, 80, 92])
marks.plot(kind="hist", bins=5, edgecolor="black",
           title="Distribution of marks")
plt.xlabel("Marks")
plt.show()

# Bar chart: compare category values
df.plot(x="month", y="sales", kind="bar",
        legend=False, title="Sales by month")
plt.ylabel("Sales")
plt.tight_layout()
plt.show()
```

3.5 When Pandas plotting is useful

• Quick exploratory charts directly from a DataFrame.

• Simple charts with minimal code.

• Fast comparison of columns or groups after data cleaning.

• Use Seaborn or direct Matplotlib when you need more specialized statistical styling or detailed control.

4. Line plots

A **line plot** connects observations in order, usually to show a trend. It is especially useful for time series such as daily temperature, monthly sales, or yearly population.

4.1 Example

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May"]
temperature = [24, 26, 29, 32, 34]

plt.plot(months, temperature, marker="o",
         color="tomato", linewidth=2)
plt.title("Monthly temperature")
plt.xlabel("Month")
plt.ylabel("Temperature (°C)")
plt.grid(True, alpha=0.25)
plt.tight_layout()
plt.show()
```

4.2 How to read it

• X-axis: usually time or an ordered sequence.

• Y-axis: the measured quantity.

• Upward sections indicate increases; downward sections indicate decreases.

• Steep sections indicate faster changes relative to the chosen scales.

• Markers show the actual observations; the connecting line guides the eye.

4.3 Multiple lines

```
days = [1, 2, 3, 4, 5]
city_a = [30, 31, 29, 32, 33]
city_b = [27, 28, 28, 29, 30]

plt.plot(days, city_a, marker="o", label="City A")
plt.plot(days, city_b, marker="s", label="City B")
plt.title("Daily temperature comparison")
plt.xlabel("Day")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.show()
```

4.4 Advantages and limitations

• Advantages: reveals trends, direction, turning points, and changes over time.

• Limitations: too many lines can clutter a chart; connecting unordered categories may imply a sequence that does not exist.

• Tip: sort time values correctly and handle missing dates intentionally before plotting.

5. Bar plots

A **bar plot** compares a numerical value across discrete categories. The length or height of each bar represents the value. Bar charts are suitable for counts, totals, averages, or other summarized measures.

5.1 Vertical and horizontal bars

```
categories = ["Python", "Java", "SQL"]
students = [35, 25, 30]

plt.bar(categories, students, color="steelblue")
plt.title("Students choosing each topic")
plt.xlabel("Topic")
plt.ylabel("Number of students")
plt.show()

# Horizontal bars are useful for long category names
plt.barh(categories, students)
plt.xlabel("Number of students")
plt.title("Topic preference")
plt.show()
```

5.2 Grouped bars

```
import numpy as np
labels = ["A", "B", "C"]
class_1 = [70, 82, 75]
class_2 = [74, 78, 88]
x = np.arange(len(labels))
width = 0.35

fig, ax = plt.subplots()
ax.bar(x - width/2, class_1, width, label="Class 1")
ax.bar(x + width/2, class_2, width, label="Class 2")
ax.set_xticks(x, labels)
ax.set_ylabel("Average marks")
ax.set_title("Class comparison")
ax.legend()
fig.tight_layout()
plt.show()
```

5.3 Bar plot versus histogram

A bar plot compares separate categories; the bars can be reordered and gaps are often shown. A histogram groups a continuous numeric variable into intervals (bins), so the order and bin boundaries matter and bars normally touch.

5.4 Good practices

• Start a bar chart's value axis at zero in most cases, because bar length encodes magnitude.

• Sort bars when ranking is the goal.

• Use horizontal bars for long labels.

• Do not use a bar chart for every numeric distribution; use a histogram when the question is about the distribution of continuous values.

6. Histograms and density plots

6.1 Histogram

A **histogram** divides numeric values into intervals called **bins** and counts how many observations fall in each bin. It helps reveal shape, spread, skewness, clusters, and possible outliers.

```
marks = [45, 50, 52, 55, 58, 60, 61, 65, 70, 72, 80, 92]

plt.hist(marks, bins=5, edgecolor="black")
plt.title("Distribution of exam marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

Bin choice matters: too few bins can hide structure; too many bins can make random fluctuations look meaningful. Try a few reasonable settings and explain your choice.

6.2 Frequency, probability and density

• Frequency: the number of observations in each bin.

• Probability-style histogram: bars are scaled so their heights represent proportions under the chosen binning.

• Density histogram: the total area of all bars is 1; density height is not the same as raw count.

```
plt.hist(marks, bins=5, density=True, alpha=0.6,
         edgecolor="black")
plt.title("Density-scaled histogram")
plt.ylabel("Density")
plt.show()
```

6.3 Density plot / KDE

A **kernel density estimate (KDE)** is a smooth estimate of a variable's distribution. It can make the overall shape easier to see, but it is an estimate—not a set of exact counts. Its appearance depends on the bandwidth (smoothing level).

```
import seaborn as sns

sns.kdeplot(data=marks, fill=True)
plt.title("KDE of exam marks")
plt.xlabel("Marks")
plt.show()
```

6.4 Histogram with density curve

```
sns.histplot(data=marks, bins=5, stat="density",
             kde=True, edgecolor="white")
plt.title("Marks: histogram and KDE")
plt.xlabel("Marks")
plt.show()
```

6.5 Shape vocabulary

• Symmetric: left and right sides look roughly alike.

• Right-skewed: a long tail extends toward larger values.

• Left-skewed: a long tail extends toward smaller values.

• Bimodal: two prominent peaks; this may indicate two subgroups.

• Outlier: an observation far from most of the data; investigate it rather than automatically deleting it.

Important: KDE curves can extend beyond the observed data range and can smooth away small groups. Use a histogram as a complementary view, especially for small datasets.

7. Scatter plots

A **scatter plot** displays paired numeric values as points. Each point represents one observation, with one variable on the x-axis and another on the y-axis. It is used to inspect association, clusters, outliers, and nonlinear patterns.

7.1 Basic scatter plot

```
hours = [1, 2, 2.5, 3, 4, 5, 6]
marks = [42, 48, 55, 58, 68, 75, 85]

plt.scatter(hours, marks, alpha=0.8)
plt.title("Study hours and exam marks")
plt.xlabel("Study hours")
plt.ylabel("Exam marks")
plt.show()
```

7.2 Interpret the pattern

• Positive association: larger x values tend to occur with larger y values.

• Negative association: larger x values tend to occur with smaller y values.

• Little visible association: no clear pattern is apparent.

• Nonlinear association: the relationship follows a curve rather than a straight trend.

• Clusters: points form groups that may represent different populations or conditions.

• Outliers: points far from the main cloud deserve investigation.

7.3 Add a third variable

```
size = [30, 50, 70, 40, 90, 60, 80]
group = ["A", "A", "B", "B", "A", "B", "A"]
colors_by_group = ["tab:blue" if g == "A" else "tab:orange"
                   for g in group]

plt.scatter(hours, marks, s=size, c=colors_by_group, alpha=0.7)
plt.xlabel("Study hours")
plt.ylabel("Exam marks")
plt.title("Scatter plot with group color and point size")
plt.show()
```

Color can encode a category, and point size can encode another numeric measure. Add a legend or clear explanation when using visual encodings; otherwise readers may not know what the colors or sizes mean.

7.4 Correlation is not causation

A visible relationship does not prove that one variable causes the other. A third factor may influence both, or the pattern may arise by chance. Scatter plots are exploratory tools, not causal proof.

7.5 Overplotting

When many points overlap, the plot can hide how many observations are present. Reduce marker size, use transparency (`alpha`), or consider a hexbin / 2D density plot for very large datasets.

8. Seaborn: statistical visualization

Seaborn is a statistical visualization library built on Matplotlib. It works naturally with Pandas DataFrames, offers attractive default styles, and makes it easy to map columns to visual properties such as color, marker style, or row/column panels.

8.1 Setup and long-form data

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "month": ["Jan", "Feb", "Mar", "Jan", "Feb", "Mar"],
    "sales": [10, 14, 13, 8, 11, 12],
    "region": ["North", "North", "North", "South", "South", "South"]
})
```

In **long-form data**, each row is one observation and columns identify variables. This layout often works well with Seaborn because the chart can map columns directly to x, y, hue, and other visual roles.

8.2 Seaborn syntax

```
sns.lineplot(data=df, x="month", y="sales", hue="region",
             marker="o")
plt.title("Sales by month and region")
plt.show()
```

`data` supplies the DataFrame; `x` and `y` name columns; `hue` splits or colors the data by a category. Seaborn may aggregate repeated observations and may display uncertainty intervals depending on the function and settings.

8.3 Common Seaborn functions

Function	Purpose
sns.lineplot()	Trend or relationship along an ordered x-axis
sns.barplot()	Estimate a numeric statistic for each category (often mean)

Function	Purpose
sns.countplot()	Count observations in each category
sns.histplot()	Histogram, optionally with KDE
sns.kdeplot()	Smoothed density estimate
sns.scatterplot()	Relationship between two numeric variables
sns.boxplot()	Compare distributions and potential outliers by category
sns.violinplot()	Show distribution shape by category
sns.stripplot()	Show individual observations by category
sns.pairplot()	Explore pairwise relationships among several variables
sns.heatmap()	Display values in a matrix, often a correlation matrix

9. Facet grids

A **facet grid** splits a dataset into multiple small charts, one for each category or combination of categories. This makes it easier to compare groups while keeping the same chart design.

9.1 FacetGrid example

```
tips = sns.load_dataset("tips")

g = sns.FacetGrid(tips, col="time", row="sex",
                  height=3, aspect=1.1)
g.map_dataframe(sns.scatterplot, x="total_bill",
                y="tip", alpha=0.7)
g.set_axis_labels("Total bill", "Tip")
g.set_titles(row_template="{row_name}",
             col_template="{col_name}")
plt.show()
```

The built-in Seaborn `tips` dataset is used in this example. If downloading built-in datasets is unavailable in your environment, replace it with your own DataFrame containing the columns used in the code.

9.2 relplot: a convenient figure-level function

```
sns.relplot(data=tips, x="total_bill", y="tip",
            hue="smoker", col="time", kind="scatter",
            height=3)
plt.show()
```

`relplot()` can create relational plots and split them across facets. `col` creates columns of panels, while `row` creates rows. `hue` distinguishes groups by color.

9.3 Benefits and cautions

• Benefits: side-by-side comparisons, visible group differences, less overplotting than combining every group into one chart.

• Keep axes comparable when the purpose is to compare magnitudes.

• Too many categories create too many tiny panels.

• Check sample sizes: a pattern in a group with very few observations may be unstable.

10. Categorical data visualization

Categorical variables contain labels or groups, such as department, product type, region, or pass/fail status. Categorical plots compare categories or show the distribution of numeric values within each category.

10.1 Count plot

```
students = pd.DataFrame({
    "course": ["Python", "Java", "Python", "SQL",
               "Java", "Python", "SQL", "Python"]
})
sns.countplot(data=students, x="course")
plt.title("Number of students by course")
plt.show()
```

A count plot counts rows in each category. It is useful when the raw data contains one row per observation and you want category frequencies.

10.2 Bar plot for a summary statistic

```
scores = pd.DataFrame({
    "class": ["A", "A", "A", "B", "B", "B"],
    "marks": [70, 75, 80, 65, 78, 82]
})
sns.barplot(data=scores, x="class", y="marks", errorbar=None)
plt.title("Average marks by class")
plt.ylabel("Mean marks")
plt.show()
```

Unlike `countplot()`, `barplot()` summarizes a numeric variable for each category. By default, it commonly estimates a mean and can show uncertainty intervals; the exact options depend on the Seaborn version.

10.3 Box plot

```
sns.boxplot(data=scores, x="class", y="marks")
plt.title("Marks distribution by class")
plt.show()
```

• The line inside the box is the median.

• The box spans the first to third quartiles (the interquartile range, IQR).

• Whiskers extend according to the plotting convention; in Seaborn's usual box plot, they typically reach the most extreme points within 1.5 × IQR of the quartiles.

• Points beyond the whiskers are potential outliers, not automatically errors.

10.4 Violin and strip plots

```
sns.violinplot(data=scores, x="class", y="marks", inner="quartile")
plt.title("Distribution shape by class")
plt.show()

sns.stripplot(data=scores, x="class", y="marks", jitter=True)
plt.title("Individual marks by class")
plt.show()
```

A violin plot shows a smoothed distribution shape for each category. A strip plot displays individual observations. With small datasets, a strip plot can be more transparent; with larger datasets, combine or overlay plots thoughtfully.

10.5 Choosing the right categorical plot

• Category frequency: count plot.

• Compare a numeric summary across categories: bar plot.

• Compare median, spread, and potential outliers: box plot.

• Compare distribution shapes: violin plot.

• Show every individual observation: strip or swarm plot.

11. Other Python visualization tools

Matplotlib, Pandas, and Seaborn are not the only options. The best tool depends on whether you need interactive exploration, browser-based dashboards, maps, or a specialized statistical display.

11.1 Plotly

Plotly creates interactive charts with hover details, zooming, and legends that users can toggle. It is useful for interactive reports and web applications.

```
# Install: python -m pip install plotly
import plotly.express as px
import pandas as pd

df = pd.DataFrame({
    "month": ["Jan", "Feb", "Mar"],
    "sales": [120, 150, 135]
})
fig = px.line(df, x="month", y="sales",
              markers=True, title="Monthly sales")
fig.show()
```

11.2 Bokeh

Bokeh supports interactive browser-based visualizations and linked charts. It can be useful when an application needs interactive tools and a Python-centered workflow.

```
# Install: python -m pip install bokeh
from bokeh.plotting import figure, show

p = figure(title="Simple Bokeh line", x_axis_label="x",
           y_axis_label="y")
p.line([1, 2, 3], [2, 5, 3], line_width=2)
show(p)
```

11.3 Altair

Altair is a declarative visualization library. You describe how data fields map to x, y, color, and marks rather than specifying every drawing step. It is concise for many statistical charts.

```
# Install: python -m pip install altair
import altair as alt
import pandas as pd

df = pd.DataFrame({
    "month": ["Jan", "Feb", "Mar"],
    "sales": [120, 150, 135]
})
chart = alt.Chart(df).mark_line(point=True).encode(
    x="month:N", y="sales:Q"
).properties(title="Monthly sales")
chart
```

11.4 Other tools to know

• **HoloViews:** high-level plotting built around data and visualization objects; can work with multiple plotting backends.

• **Datashader:** helps render very large datasets by aggregating data into pixels, reducing overplotting.

• **GeoPandas / Folium:** useful for geospatial data and interactive maps.

• **Dash / Streamlit:** application frameworks for building dashboards around charts and controls.

• **Pygal:** Python charting library focused on SVG output; check project maintenance and compatibility before choosing it for a new project.

11.5 Quick comparison

Tool	Main strength	Best fit
Matplotlib	Fine-grained control	Publication-quality static plots, custom layouts
Pandas plotting	Very quick from tables	Exploratory charts
Seaborn	Statistical plots and concise syntax	Distributions, relationships, categories
Plotly	Interactive charts	Interactive analysis and web reports

Tool	Main strength	Best fit
Bokeh	Browser-based interaction	Interactive applications
Altair	Declarative grammar	Concise data-driven charts
GeoPandas / Folium	Geospatial visualization	Maps and spatial data

12. A complete mini example

This example puts several topics together: a DataFrame, a line plot, a histogram, a scatter plot, and a categorical comparison. Run each section separately if you want to study one chart at a time.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.DataFrame({
    "student": ["A", "B", "C", "D", "E", "F"],
    "hours": [1, 2, 2.5, 3, 4, 5],
    "marks": [42, 48, 55, 58, 68, 75],
    "group": ["X", "X", "Y", "Y", "X", "Y"]
})

# 1. Scatter plot: study hours versus marks
sns.scatterplot(data=df, x="hours", y="marks", hue="group")
plt.title("Study hours and marks")
plt.tight_layout()
plt.show()

# 2. Histogram: distribution of marks
sns.histplot(data=df, x="marks", bins=5, kde=True)
plt.title("Distribution of marks")
plt.tight_layout()
plt.show()

# 3. Categorical summary: marks by group
sns.boxplot(data=df, x="group", y="marks")
plt.title("Marks by group")
plt.tight_layout()
plt.show()
```

Notice that the same dataset can support different questions. The scatter plot asks about the relationship between two numeric variables; the histogram asks how marks are distributed; the box plot compares marks across groups.

13. Common mistakes and debugging

• **Missing import:** use `import matplotlib.pyplot as plt` and import Pandas/Seaborn when needed.

• **Wrong column name:** check spelling and capitalization with `print(df.columns)`.

• **Chart not visible in a script:** add `plt.show()`.

• **Labels cut off:** use `plt.tight_layout()` or `fig.tight_layout()`.

• **Unexpected order:** explicitly sort dates or define category order.

• **Wrong plot type:** use bars for categories and histograms for numeric distributions.

• **Too many overlapping points:** reduce marker size, add transparency, or aggregate the data.

• **Misleading scale:** check axis limits and units; avoid visually exaggerating differences.

• **Seaborn version mismatch:** some parameters differ between versions; inspect the error and your installed version before changing the whole program.

14. Quick revision sheet

Concept	Remember
Figure	The whole canvas containing the plot(s).
Axes	The area where data is plotted; has x/y axes.
Line plot	Trend or change along an ordered sequence, often time.
Bar plot	Comparison across discrete categories.
Histogram	Frequency or density across numeric bins.
KDE / density plot	Smoothed estimate of a distribution.
Scatter plot	Relationship between two numeric variables.
Facet grid	Small multiples split by categories.
hue	Maps a variable to color to distinguish groups.
countplot	Counts observations in categories.
barplot	Summarizes a numeric variable by category.
boxplot	Median, quartiles, spread, and potential outliers.
plt.show()	Displays a plot in a script.
fig.savefig()	Saves a figure to a file.

15. Practice questions

1. What is data visualization? Give three benefits.

2. Explain the difference between a Figure and an Axes in Matplotlib.

3. Write Python code to create a line plot with a title and labeled axes.

4. When would you choose a bar plot instead of a histogram?

5. What are bins in a histogram? How can the number of bins change the appearance?

6. What is the difference between frequency and density in a histogram?

7. What does a KDE plot show, and why is its shape an estimate?

8. How can you interpret positive, negative, and nonlinear patterns in a scatter plot?

9. What does the Seaborn parameter hue do?

10. Explain FacetGrid and give one situation where it is useful.

11. Differentiate countplot() and barplot().

12. Compare Matplotlib, Pandas plotting, Seaborn, and Plotly.

13. Name two ways to reduce overplotting in a scatter plot.

14. Why does correlation not necessarily imply causation?

15. List four common chart design practices that improve readability.

Short answer key

1. Visual representation of data; helps reveal patterns, compare groups, communicate results, and spot anomalies. 2. Figure is the full canvas; Axes is a plotting area. 3. Use `plt.plot()`, `plt.title()`, `plt.xlabel()`, `plt.ylabel()`, and `plt.show()`. 4. Bar plot for discrete categories; histogram for the distribution of numeric values. 5. Bins are intervals; too few hide detail and too many may add noise. 6. Frequency is count; density scales the total histogram area to one. 7. KDE is a smoothed estimate whose shape depends on bandwidth. 8. Upward, downward, or

curved patterns indicate positive, negative, or nonlinear association. 9. Maps a variable to color. 10. A grid of small charts split by categories. 11. Countplot counts rows; barplot summarizes a numeric variable. 12. Matplotlib gives control, Pandas is quick, Seaborn supports statistical plots, and Plotly adds interactivity. 13. Smaller markers, transparency, or aggregation. 14. Confounding factors or chance can explain an association. 15. Use clear titles, labeled axes with units, readable scales, and legends only when needed.

16. Suggested practical exercises

1. Create a line plot of monthly expenses and label the y-axis in rupees.
2. Create a bar chart comparing marks in five subjects; display the subjects from highest to lowest.
3. Generate 100 random marks and compare histograms with 5, 10, and 20 bins.
4. Create a scatter plot of study hours and marks; color points by class group.
5. Use a box plot to compare marks across three classes.
6. Use FacetGrid or relplot() to compare a relationship across two categories.
7. Save one chart as a PNG file using 300 DPI and tight bounding.

End of Module 3 notes. For exams, practice writing short definitions, explaining the purpose of each plot, comparing similar chart types, and reproducing small code examples.